

Trabalho Prático 02: S-AES e AES/Modos de Operação

Leia o [README.md](#)

O trabalho e o código estão disponíveis neste [repositório](#)

Identificação:

- *Aluno:* Willyan Marques de Melo
- *Matrícula:* 221020940
- *Professor(a):* Lorena de Souza Bezerra Borges
- *Turma:* CIC0201 - Turma 2

Objetivo

Desenvolver uma implementação do Simplified AES (S-AES), utilizando blocos de 16 bits e composto por 2 rodadas, seguindo o funcionamento do AES original.

Introdução

O Advanced Encryption Standard (AES) é um dos principais algoritmos de criptografia simétrica utilizados atualmente e foi criado pelo NIST (National Institute of Standards and Technology) em 2001 como uma nova alternativa para o DES (Data Encryption Standard) que se tornou inseguro. O AES é um algoritmo seguro e eficiente que opera em blocos de 128 bits e tem variações com chaves de 128, 192 e 256 bits. De forma análoga ao DES, ele executa diversas rodadas que fazem substituições, permutações e operações aritméticas no campo finito $GF(2^8)$ em matrizes de bytes.

Para fins didáticos, o Simplified AES (S-AES) foi desenvolvido como uma versão menos complexa do AES. O S-AES utiliza blocos de 16 bits e chaves de 16 bits, realiza apenas 2 rodadas e as operações no campo finito são realizadas no campo $GF(2^4)$. Apesar de sua simplicidade, o S-AES mantém a mesma estrutura do AES, como substituições com S-box, permutações e operações no campo finito. Por isso, ele é utilizado como forma de ensinar os princípios básicos de funcionamento e implementação do algoritmo original. Mesmo assim, ambos compartilham a mesma filosofia de design, baseada em múltiplas camadas de transformação para garantir segurança contra ataques.

Desenvolvimento

A classe `Bin`

A classe `Bin` foi projetada para representar e manipular valores binários de tamanho fixo em Python. Ela encapsula um valor inteiro (`value`) e seu tamanho em bits (`size`), oferecendo diversas operações e conversões úteis para realizar álgebra booleana, manipulação de bits e codificações.

Principais componentes

- **Inicialização**
 - `__init__(self, value, size)`: Inicializa um objeto `Bin` com o valor inteiro e o tamanho especificado em bits.
- **Representações e formatação**
 - `__repr__` e `__str__`: Retornam representações em hexadecimal para debug e exibição.
 - `__format__`: Permite formatar o objeto em diferentes bases (binária, hexadecimal, base64) com suporte a agrupamento de dígitos.
- **Operações bit a bit**
 - `__xor__`, `__or__`, `__and__`: Realizam operações lógicas bit a bit entre dois objetos `Bin`, garantindo que ambos tenham o mesmo tamanho.
 - `__lshift__`, `__rshift__`: Realizam deslocamentos circulares dos bits para a esquerda ou direita.
 - `xor(self, *bins)`: Aplica XOR sucessivo entre o objeto e múltiplos `Bin`.
- **Operações aritméticas**
 - `__add__`, `__mul__`: Permitem somar e multiplicar dois objetos `Bin`, ajustando o tamanho do resultado de acordo com o número de bits necessários.
 - `finite_field_mul`: Realiza multiplicação no campo finito $GF(2^4)$, com redução modular pelo polinômio $0x13$ (útil em criptografia, como no AES).
- **Manipulação e transformação**
 - `copy`: Cria uma cópia do objeto.
 - `to_list`: Converte o valor em uma lista de bits.
 - `to_bytes`: Converte o valor para um array de bytes.
 - `to_str`: Converte o valor para uma string em formato binário, hexadecimal ou base64, com agrupamento opcional.
 - `extract`: Extrai bits em posições específicas, retornando um novo `Bin`.
 - `halve`: Divide o `Bin` em duas metades.
 - `split`: Divide o valor em pedaços menores de tamanho fixo.
 - `swap`: Troca as metades esquerda e direita do valor.
 - `extend`: Anexa os bits de outro `Bin` ao final do valor atual.
 - `fuse`: Junta vários objetos `Bin` em um único.
 - `from_hex`: Cria um `Bin` a partir de uma string hexadecimal.
- **Validação**
 - `assert_same_size`: Verifica se dois objetos `Bin` têm o mesmo tamanho antes de operações bit a bit.

```
import base64
from typing import List

# Representa um número binário com tamanho fixo
class Bin:
    def __init__(self, value: int, size: int):
        self.value = value # Valor inteiro
```

```

        self.size = size # Tamanho em bits

# Representação para debug (em hexadecimal)
def __repr__(self):
    return f"Bin(value={self}, size={self.size})"

# Representação em string (em hexadecimal)
def __str__(self, type="x", group_size=0):
    hex_digits = (self.size + 3) // 4
    return f"0x{self.value:0{hex_digits}X}"

# Format_spec example: "x", "b", or "b_4" (for grouping)
def __format__(self, format_spec):
    if not format_spec:
        return str(self)
    parts = format_spec.split("_")
    fmt = parts[0]
    group_size = int(parts[1]) if len(parts) > 1 else 0

    if fmt in ("b", "B", "x", "X", "base64"):
        return self.to_str(fmt, group_size)
    else:
        return str(self)

# Verifica igualdade com outro objeto Bin
def __eq__(self, other):
    if isinstance(other, Bin):
        return self.value == other.value and self.size ==
other.size
    return False

# Verifica desigualdade com outro objeto Bin
def __ne__(self, other):
    if isinstance(other, Bin):
        return not self.__eq__(other)
    return True

# Faz o XOR bit a bit entre dois objetos Bin
def __xor__(self, other: "Bin") -> "Bin":
    self.assert_same_size(other)
    return Bin(self.value ^ other.value, self.size)

# Faz o OR bit a bit entre dois objetos Bin
def __or__(self, other: "Bin") -> "Bin":
    self.assert_same_size(other)
    return Bin(self.value | other.value, self.size)

# Faz o AND bit a bit entre dois objetos Bin
def __and__(self, other: "Bin") -> "Bin":
    self.assert_same_size(other)

```

```

        return Bin(self.value & other.value, self.size)

# Faz o MUL entre dois objetos Bin
def __mul__(self, other: "Bin") -> "Bin":
    result_value = self.value * other.value
    result_size = result_value.bit_length() or 1
    return Bin(result_value, result_size)

# Faz o ADD entre dois objetos Bin
def __add__(self, other: "Bin") -> "Bin":
    result_value = self.value + other.value
    result_size = result_value.bit_length() or 1
    return Bin(result_value, result_size)

# Faz o circular shift à esquerda
def __lshift__(self, shift_value: int) -> "Bin":
    shift_value %= self.size
    discarded = self.value >> (self.size - shift_value)
    mask = (1 << self.size) - 1
    shifted = (self.value << shift_value) & mask
    result = shifted | discarded
    return Bin(result, self.size)

# Faz o circular shift à direita
def __rshift__(self, shift_value: int) -> "Bin":
    shift_value %= self.size
    discarded = (self.value << (self.size - shift_value)) & ((1 <<
self.size) - 1)
    shifted = self.value >> shift_value
    result = shifted | discarded
    return Bin(result, self.size)

# Faz a cópia do objeto Bin
def copy(self):
    return Bin(self.value, self.size)

# Converte o valor Bin para uma lista de bits
def to_list(self) -> List[int]:
    return [(self.value >> i) & 1 for i in
reversed(range(self.size))]

def to_bytes(self):
    byte_length = (self.size + 7) // 8
    return self.value.to_bytes(byte_length, byteorder='big')

# Converte o valor Bin para uma string formatada
def to_str(self, fmt="x", group_size=0):
    if fmt in ("x", "X"):
        hex_digits = (self.size + 3) // 4
        raw_hex = f"{self.value:0{hex_digits}{fmt}}"

```

```

        if group_size <= 0:
            return f"0x{raw_hex}"
        else:
            # Group hex digits from right to left (common style)
            groups = []
            for i in range(len(raw_hex), 0, -group_size):
                start = max(0, i - group_size)
                groups.append(raw_hex[start:i])
            grouped = "_".join(reversed(groups))
            return f"0x{grouped}"

    elif fmt in ("b", "B"):
        raw_bin = f"{self.value:0{self.size}b}"
        if group_size <= 0:
            return f"0b{raw_bin}"
        else:
            grouped = "_".join(
                raw_bin[i : i + group_size]
                for i in range(0, len(raw_bin), group_size)
            )
            return f"0b{grouped}"

    elif fmt == "base64":
        raw_bytes = self.to_bytes()
        b64_str = base64.b64encode(raw_bytes).decode("ascii")
        if group_size <= 0:
            return b64_str
        else:
            grouped = " ".join(
                b64_str[i : i + group_size]
                for i in range(0, len(b64_str), group_size)
            )
            return grouped
    else:
        raise ValueError(f"Unsupported format: {fmt}")

# Realiza operação XOR bit a bit entre vários Bin
    def xor(self, *bins: "Bin"):
        for bin in bins:
            self.assert_same_size(bin)
            self.value ^= bin.value
        return self

# Extrai bits com base em uma tabela de posições e retorna como novo Bin
    def extract(self, table: List[List[int]]):
        extracted_bits = []
        for row in table:
            for bit_pos in row:
                bit = (self.value >> (self.size - bit_pos)) & 1

```

```

        extracted_bits.append(bit)
    result = 0
    for bit in extracted_bits:
        result = (result << 1) | bit
    return Bin(result, len(extracted_bits))

# Divide o valor Bin em duas metades e retorna como dois objetos
Bin
def halve(self):
    half = (self.size + 1) // 2
    left = (self.value >> half) & ((1 << half) - 1)
    right = self.value & ((1 << half) - 1)
    return Bin(left, half), Bin(right, half)

# Divide o Bin em pedaços menores de tamanho fixo
def split(self, chunk_size: int):
    remainder = self.size % chunk_size
    if remainder != 0:
        # Calculate how many bits to pad
        pad_size = chunk_size - remainder
        self.value <=< pad_size # Shift left to pad with zeros
        self.size += pad_size

    chunks: List[Bin] = []
    for i in range(0, self.size, chunk_size):
        chunk_value = (self.value >> (self.size - i - chunk_size))
        & (
            (1 << chunk_size) - 1
        )
        chunks.append(Bin(chunk_value, chunk_size))
    return chunks

# Troca as metades esquerda e direita de um Bin
def swap(self):
    left, right = self.halve()
    right.extend(left)
    self.value = right.value

# Anexa os bits de outro Bin ao final deste Bin
def extend(self, bin: "Bin"):
    self.value = (self.value << bin.size) | bin.value
    self.size += bin.size

def assert_same_size(self, bin: "Bin"):
    if self.size != bin.size:
        raise ValueError("Bin sizes must match for bitwise
operations")

# Junta vários objetos Bin em um só
@staticmethod

```

```

def fuse(*bins: "Bin"):
    total_value = 0
    total_size = 0
    for b in bins:
        total_value = (total_value << b.size) | b.value
        total_size += b.size
    return Bin(total_value, total_size)

# Converte uma str em formato '0x' para um Bin
@staticmethod
def from_hex(hex_str: str):
    return Bin(int(hex_str, 16), len(hex_str) * 4)

# Multiplica dois objetos Bin em um campo finito (GF(2^4) com
# módulo 0x13)
@staticmethod
def finite_field_mul(a: "Bin", b: "Bin") -> "Bin":
    assert a.size == b.size == 4, "Operands must be 4-bit for
    GF(2^4)"

    modulus = 0x13
    result = 0
    a_val = a.value
    b_val = b.value
    for _ in range(4): # GF(2^4)
        if b_val & 1:
            result ^= a_val
        b_val >>= 1
        a_val <<= 1
        if a_val & 0x10: # Check if x^4 is set (overflow)
            a_val ^= modulus # Modular reduction

    return Bin(result & 0xF, 4) # Ensure result is 4 bits

```

A classe `BinBlock`

A classe `BinBlock` representa especificamente uma matriz 2x2 de nibbles (grupos de 4 bits), construída a partir de um valor binário de 16 bits. Essa estrutura é utilizada no corpo principal do AES para realizar operações entre os valores binários nos rounds principais e na geração de chave.

Principais componentes

- **Inicialização**
 - `__init__(self, bin=Bin(0x0, 16))`: O construtor recebe um objeto `Bin` com exatamente 16 bits. Esse valor é dividido em quatro nibbles (4 bits cada), organizados em um bloco 2x2 da seguinte forma:

```
[[nibble0, nibble2],  
 [nibble1, nibble3]]
```

Isso permite acesso e manipulação de cada pedaço de 4 bits individualmente no contexto do bloco.

- **Representações**

- `__repr__`: Fornece uma representação para debug, mostrando o valor e os tamanho da palavra binária.
- `__str__`: Retorna uma string formatada com os nibbles em hexadecimal, organizados em linhas.

- **Operadores bit a bit**

- `__xor__`, `__or__`, `__and__`: Realizam operações bit a bit entre dois blocos (BinBlock), aplicando a operação correspondente elemento a elemento. O resultado é um novo BinBlock com os valores calculados.

- **Manipulação de linhas e colunas**

- `shift_line_left(l, amt)`: Faz um deslocamento circular à esquerda em uma linha do bloco (linha `l`) por uma quantidade `amt`. A linha é primeiro fundida em um único Bin, deslocada e depois dividida novamente em nibbles.
- `get_column(c)`: Retorna a coluna `c` como uma lista de dois Bin. Obs.: O índice da coluna é limitado ao intervalo válido.
- `get_columns()`: Retorna todas as colunas do bloco como uma lista de listas, permitindo iteração coluna por coluna.

- **Outros métodos**

- `copy()`: Cria uma cópia do bloco, preservando a estrutura e os valores.
- `to_bin()`: Concatena os nibbles do bloco em um único Bin de 16 bits. A ordem dos nibbles na fusão respeita o padrão:

```
[nibble0, nibble2, nibble1, nibble3]
```

garantindo consistência com a forma como o bloco foi construído.

```
class BinBlock:  
    block: List[List[Bin]] = []  
  
    def __init__(self, bin=Bin(0x0, 16)):  
        if bin.size != 16:  
            raise ValueError("BinBlock requires a 16-bit Bin")  
  
        nibbles = bin.split(4)  
        block = [  
            [nibbles[0], nibbles[2]],  
            [nibbles[1], nibbles[3]]
```



```

    ]

    self.block = block
    self.nibbles = 4

    # Representação para debug
    def __repr__(self):
        rows = ",\n ".join([str(row) for row in self.block])
        return f"BinBlock([\n {rows}\n])"

    # Representação em string
    def __str__(self):
        rows = "\n".join([" ".join([str(b) for b in row]) for row in
self.block])
        return rows

    # Verifica igualdade com outro BinBlock
    def __eq__(self, other):
        if not isinstance(other, BinBlock):
            return False
        return self.block == other.block and self.nibbles ==
other.nibbles

    # Faz o XOR bit a bit entre dois BinBlocks
    def __xor__(self, other: "BinBlock"):
        if not isinstance(other, BinBlock):
            raise TypeError("Operand must be a BinBlock")

        result_block = []
        for row_self, row_other in zip(self.block, other.block):
            result_row = [a ^ b for a, b in zip(row_self, row_other)]
            result_block.append(result_row)

        result = BinBlock.__new__(BinBlock)
        result.block = result_block
        result.nibbles = self.nibbles
        return result

    # Faz o OR bit a bit entre dois BinBlocks
    def __or__(self, other: "BinBlock"):
        if not isinstance(other, BinBlock):
            raise TypeError("Operand must be a BinBlock")

        result_block = []
        for row_self, row_other in zip(self.block, other.block):
            result_row = [a | b for a, b in zip(row_self, row_other)]
            result_block.append(result_row)

        result = BinBlock.__new__(BinBlock)

```

```

        result.block = result_block
        result.nibbles = self.nibbles
        return result

# Faz o AND bit a bit entre dois BinBlocks
def __and__(self, other: "BinBlock"):
    if not isinstance(other, BinBlock):
        raise TypeError("Operand must be a BinBlock")

    result_block = []
    for row_self, row_other in zip(self.block, other.block):
        result_row = [a & b for a, b in zip(row_self, row_other)]
        result_block.append(result_row)

    result = BinBlock.__new__(BinBlock)
    result.block = result_block
    result.nibbles = self.nibbles
    return result

# Faz o shift circular à esquerda de uma linha específica
def shift_line_left(self, l: int, amt: int):
    line = Bin.fuse(*self.block[l])
    line = line << amt
    self.block[l] = line.split(4)

# Retorna uma coluna específica do bloco como uma lista de Bin
def get_column(self, c: int) -> List[Bin]:
    if c not in range(4):
        raise IndexError("Column index must be between 0 and 3")
    return [self.block[row][c] for row in range(2)]

# Retorna todas as colunas do bloco como uma lista de listas de Bin
def get_columns(self) -> List[List[Bin]]:
    return [[self.block[row][c] for row in range(2)] for c in range(2)]

# Retorna uma cópia do BinBlock
def copy(self) -> "BinBlock":
    new_block = BinBlock()
    new_block.block = self.block.copy()
    new_block.nibbles = self.nibbles
    return new_block

# Converte o BinBlock para um único Bin
def to_bin(self) -> Bin:
    flat = [b for row in self.block for b in row]
    ordered = [flat[0], flat[2], flat[1], flat[3]]
    return Bin.fuse(*ordered)

```

A classe `Logger`

A classe `Logger` faz o registro de logs organizados por título ou fase e é utilizada para acompanhar cada etapa do AES, facilitando a depuração do programa. O corpo principal da classe consiste em um dicionário (`self.logs`) que armazena listas de tuplas, onde cada chave é a fase e as tuplas contêm uma descrição e o objeto/valor registrado.

Principais métodos:

- `log(description: str, obj, title: str = "default"):`
Adiciona uma entrada de log sob um título específico. Cada entrada é composta por uma descrição e o objeto a ser registrado.
- `show(title: str = None):`
Exibe os logs registrados. Se um título for fornecido, mostra apenas os logs daquela fase; caso contrário, exibe todos os logs agrupados por título. Cada log é apresentado com seu índice, descrição e representação do objeto.

Essa abordagem modular permite acompanhar facilmente o fluxo de execução e analisar o estado interno do sistema em diferentes etapas, facilitando a depuração e validação de cada fase.

```
class Logger:
    def __init__(self):
        self.logs = {}

    def log(self, description: str, obj, title: str = "default"):
        if title not in self.logs:
            self.logs[title] = []
        self.logs[title].append((description, obj))

    def show(self, title: str = None):
        if title:
            logs = self.logs.get(title, [])
            print(f"=== Logs for phase '{title}' ===\n")
            for i, (desc, obj) in enumerate(logs, 1):
                print(f"[{i}] {desc}:\n{str(obj)}\n")
        else:
            for title, entries in self.logs.items():
                print(f"=== Logs for phase '{title}' ===\n")
                for i, (desc, obj) in enumerate(entries, 1):
                    print(f"[{i}] {desc}:\n{str(obj)}\n")
            print("_" * 40)
```

A classe `S_AES`

A classe `S_AES` representa uma implementação didática e simplificada do AES. Seu objetivo é encapsular as operações internas da criptografia como a geração de chaves e os modos de operação.

Principais características

- **Inicialização**
 - `__init__(self, key_size=16, mode="ECB")` Recebe um tamanho de chave (padrão: 16 bits) e um modo de operação, valida o tamanho da chave, define o número de rodadas e inicializa um objeto `Logger` para registrar o estado das operações.
- **Método principal**
 - `encrypt(plaintext: str, key: Bin, ...)`
Recebe o texto plano como string e a chave como `Bin`. Converte o texto para binário (`convert_to_bin`) e chama a função de criptografia no modo selecionado (atualmente apenas ECB).
- **Conversão de texto**
 - `convert_to_bin(plaintext: str) -> Bin`
Transforma uma string em uma representação binária (`Bin`), codificando em UTF-8 e convertendo os bytes em inteiro.
- **Criptografia de bloco**
 - `aes_ecb_encrypt(plaintext: Bin, key: Bin) -> Bin`
Divide o texto em blocos de 16 bits e aplica a criptografia bloco a bloco no modo ECB.
 - `encrypt_block(plaintext: Bin, key: Bin) -> Bin`
Criptografa um único bloco: realiza o `AddRoundKey` inicial, depois executa as rodadas principais usando o método `transform`.
- **Expansão de chave**
 - `key_expansion(key: Bin)`
Gera as subchaves para cada rodada. Utiliza a função auxiliar `g`, que rotaciona nibbles, aplica a S-box e combina com a constante de rodada (`r_con`) via XOR. As subchaves são armazenadas como blocos e registradas no logger.
- **Transformações de rodada**
 - `transform(plt: BinBlock, subkey: BinBlock, round: int)`
Executa as operações de uma rodada:
 - `SubBytes(sub_nibbles)`
 - `ShiftRows(shift_rows)`
 - `MixColumns(mix_columns, exceto na última rodada)`
 - `AddRoundKey`
- **Operações internas**
 - `sub_nibbles(plt: BinBlock) -> BinBlock`
Aplica a S-box a cada nibble do bloco.
 - `shift_rows(plt: BinBlock) -> BinBlock`
Realiza deslocamento circular na segunda linha do bloco.

- `mix_columns(plt: BinBlock) -> BinBlock`
Mistura as colunas via multiplicação no campo finito.
- `sbox(nibble: Bin) -> Bin`
Implementa a substituição usando a S-box.
- **Validação**
 - `check_key_size(key: Bin)`
Garante que a chave tem o tamanho esperado, lançando exceção caso contrário.

```

r_con = [
    0x80, 0x30
]

mix_columns_table = [
    [1, 4],
    [4, 1],
]

substitution_box = [
    [0x9, 0x4, 0xA, 0xB,],
    [0xD, 0x1, 0x8, 0x5,],
    [0x6, 0x2, 0x0, 0x3,],
    [0xC, 0xE, 0xF, 0x7,],
]

sbox_inv = [
    [0xA, 0x5, 0x9, 0xB,],
    [0x1, 0x7, 0x8, 0xF,],
    [0x6, 0x0, 0x2, 0x3,],
    [0xC, 0x4, 0xD, 0xE,],
]

class S_AES:

    def __init__(self, key_size=16, mode="ECB"):
        # Verifica se o tamanho da chave é válido
        if key_size not in [16, 128, 192, 256]:
            raise ValueError("Invalid key size for AES")

        # Define o número de rodadas conforme o tamanho da chave
        round_numbers = {16: 2, 128: 10, 192: 12, 256: 14}

        self.logger = Logger()
        self.key_size = key_size
        self.rounds = round_numbers[key_size]
        self.mode = mode

    @staticmethod
    def sbox(nibble: Bin):
        # Aplica a substituição S-Box a um nibble (4 bits)
        if nibble.size != 4:

```

```

        raise ValueError("Argument must be a nibble (4 bits)")

    l, c = [b.value for b in nibble.halve()]
    return Bin(substitution_box[l][c], 4)

def key_expansion(self, key: Bin):
    # Expande a chave para gerar as round keys
    def g(w: Bin, r: int):
        rotw = w << 4 # Rotaciona 4 bits à esquerda
        nibbles = rotw.split(4)
        nibbles = list(map(S_AES.sbox, nibbles))
        subw = Bin.fuse(*nibbles)
        rcon = Bin(r_con[r], 8)
        g_result = subw ^ rcon
        return g_result

    key_block = BinBlock(key)
    columns = key_block.get_columns()
    w0 = Bin.fuse(*columns[0])
    w1 = Bin.fuse(*columns[1])
    words: List[List[Bin]] = [[w0, w1]]

    round_keys = [BinBlock(Bin.fuse(*words[0]))]
    self.logger.log(f"Round Key 0", round_keys[0], title="Key
Expansion")
    for i in range(2):
        w0, w1 = words[i]
        w2 = w0 ^ g(w1, i)
        w3 = w2 ^ w1

        words.append([w2, w3])
        subkey = BinBlock(Bin.fuse(w2, w3))
        round_keys.append(subkey)
        self.logger.log(f"Round Key {i + 1}", subkey, title="Key
Expansion")
    return round_keys

    @staticmethod
    def sub_nibbles(plt: BinBlock) -> BinBlock:
        # Aplica S-Box a cada nibble do bloco
        block = plt.block
        cipher_block = BinBlock()
        for i in range(2):
            for j in range(2):
                s = S_AES.sbox(block[i][j])
                cipher_block.block[i][j] = s

        return cipher_block

    @staticmethod

```

```

def shift_rows(plt: BinBlock):
    # Desloca a segunda linha do bloco para a esquerda
    cipher_block = plt.copy()
    cipher_block.shift_line_left(1, 4)
    return cipher_block

@staticmethod
def mix_columns(plt: BinBlock) -> BinBlock:
    # Mistura as colunas do bloco usando multiplicação em campo
    finito
    r = Bin(0, 0)
    mct = [[Bin(k, 4) for k in l] for l in mix_columns_table]

    for c in plt.get_columns():
        for l in mct:
            nibble = []
            for i in range(2):
                nibble.append(Bin.finite_field_mul(l[i], c[i]))
            r.extend(Bin.xor(*nibble))

    return BinBlock(r)

def transform(self, plt: BinBlock, subkey: BinBlock, round: int):
    # Executa as transformações de uma rodada
    last_round = round == self.rounds
    title = f"round {round}"
    self.logger.log("Round key", subkey, title=title)
    self.logger.log("Before Transformations", plt, title=title)

    block = S_AES.sub_nibbles(plt)
    self.logger.log("After SubBytes", block, title=title)

    block = S_AES.shift_rows(block)
    self.logger.log("After ShiftRows", block, title=title)

    if not last_round:
        block = S_AES.mix_columns(block)
        self.logger.log("After MixColumns", block, title=title)

    block = block ^ subkey
    self.logger.log("After AddRoundKey", block, title=title)
    return block

def encrypt_block(self, plaintext: Bin, key: Bin) -> Bin:
    # Converte para um bloco de 16 bits
    plt_block = BinBlock(plaintext)
    rnd_keys = self.key_expansion(key)

    # Rodada inicial
    cipher_block = plt_block ^ rnd_keys[0]

```

```

        self.logger.log("Plaintext block", plt_block, title="round 0")
        self.logger.log("Initial round key", rnd_keys[0], title="round
0")
        self.logger.log("After AddRoundKey", cipher_block,
title="round 0")

        # Rodadas principais
        for i in range(1, self.rounds + 1):
            cipher_block = self.transform(cipher_block, rnd_keys[i],
i)

        return cipher_block.to_bin()

def decrypt_block(self, ciphertext: Bin, key: Bin) -> Bin:
    # Método de descryptografia ainda não implementado
    pass

def aes_ecb_encrypt(self, plaintext: Bin, key: Bin) -> Bin:
    # Criptografa o texto em blocos no modo ECB
    chunks = plaintext.split(16)
    blocks: List[Bin] = []

    for chunk in chunks:
        blocks.append(self.encrypt_block(chunk, key))

    ciphertext = Bin.fuse(*blocks)
    return ciphertext

def convert_to_bin(self, plaintext: str) -> Bin:
    # Converte string para Bin (binário)
    plaintext_bytes = plaintext.encode('utf-8')
    plaintext_int = int.from_bytes(plaintext_bytes,
byteorder='big')
    plaintext_size = len(plaintext_bytes) * 8
    plaintext_bin = Bin(plaintext_int, plaintext_size)
    return plaintext_bin

def encrypt(self, plaintext: str, key: Bin, iv: Bin = None,
key_size=16, mode="ECB") -> Bin:
    # Interface de alto nível para criptografar uma string
    self.check_key_size(key)
    plaintext_bin = self.convert_to_bin(plaintext)
    return self.aes_ecb_encrypt(plaintext_bin, key)

def check_key_size(self, key: Bin):
    # Verifica se o tamanho da chave está correto
    if key.size != self.key_size:
        raise ValueError(
            f"Key does not match the expected size for AES-

```



```
{self.key_size}"  
    )
```

Para validar a implementação do S-AES, foi realizado um teste simples com uma chave e o texto dados pelo roteiro. Uma instância do S-AES foi criada e utilizada para criptografar o texto "ok", com a chave 0xA73B e resultado da criptografia foi comparado a um valor esperado (0x0738). Os resultados obtidos foram exibidos nos formatos hexadecimal e em base64 como demandado. Além disso, também foi feito um teste para identificar repetição de padrões nos blocos de cifra gerados no modo de operação ECB. Por fim, foram exibidos os logs gerados durante a criptografia da mensagem, de forma que é possível acompanhar passo a passo da cifra.

```
# Cria instância do AES  
aes = S_AES()  
  
# Exemplo de chave e texto puro  
example_key = Bin(0xA73B, 16)  
example_plt = "ok"  
expected_ciphertext = Bin(0x0738, 16)  
  
# Realiza a criptografia  
obtained_ciphertext = aes.encrypt(example_plt, example_key)  
  
# Exibe os resultados em diferentes formatos  
print("=== RESULTADOS DO TESTE ===")  
print(f"Esperado (Hex)      : {expected_ciphertext}")  
print(f"Obtido (Hex)       : {obtained_ciphertext}")  
print(f"Esperado (base64): {format(expected_ciphertext, 'base64')}")  
print(f"Obtido (base64): {format(obtained_ciphertext, 'base64')}")  
  
# Realiza asserção com mensagem clara  
assert obtained_ciphertext == expected_ciphertext, (  
    f"Falha no teste: Ciphertext diferente!\n"  
    f"Esperado: {expected_ciphertext}\nObtido: {obtained_ciphertext}"  
)  
  
aes2 = S_AES()  
# Exemplo de chave e dois blocos de texto puro iguais  
block1 = "will"  
block2 = "willlyan marques will"  
  
# Criptografa os dois blocos  
cipher1 = aes2.encrypt(block1, example_key)  
cipher2 = aes2.encrypt(block2, example_key)  
  
# Exibe os resultados  
print("\n=== COMPARAÇÃO DE BLOCOS NO MODO ECB ===")  
print(f"Bloco 1 ciphertext: {cipher1}")  
print(f"Bloco 2 ciphertext: {cipher2}")
```

```
# Exibe os logs (se houver)
print(" " * 40)
print("\n=== LOGS GERADOS ===")
aes.logger.show()

=== RESULTADOS DO TESTE ===
Esperado (Hex)    : 0x0738
Obtido  (Hex)    : 0x0738
Esperado (base64): Bzg=
Obtido  (base64): Bzg=

=== COMPARAÇÃO DE BLOCOS NO MODO ECB ===
Bloco 1 ciphertext: 0x36127F2D
Bloco 2 ciphertext: 0x36127F2D0A553EA19DE30365F4BEC40B36127F2D



---



=== LOGS GERADOS ===
=== Logs for phase 'Key Expansion' ===

[1] Round Key 0:
0xA 0x3
0x7 0xB

[2] Round Key 1:
0x1 0x2
0xC 0x7

[3] Round Key 2:
0x7 0x5
0x6 0x1



---



=== Logs for phase 'round 0' ===

[1] Plaintext block:
0x6 0x6
0xF 0xB

[2] Initial round key:
0xA 0x3
0x7 0xB

[3] After AddRoundKey:
0xC 0x5
0x8 0x0



---



=== Logs for phase 'round 1' ===

[1] Round key:
```

0x1 0x2
0xC 0x7

[2] Before Transformations:
0xC 0x5
0x8 0x0

[3] After SubBytes:
0xC 0x1
0x6 0x9

[4] After ShiftRows:
0xC 0x1
0x9 0x6

[5] After MixColumns:
0xE 0xA
0xC 0x2

[6] After AddRoundKey:
0xF 0x8
0x0 0x5

=== Logs for phase 'round 2' ===

[1] Round key:
0x7 0x5
0x6 0x1

[2] Before Transformations:
0xF 0x8
0x0 0x5

[3] After SubBytes:
0x7 0x6
0x9 0x1

[4] After ShiftRows:
0x7 0x6
0x1 0x9

[5] After AddRoundKey:
0x0 0x3
0x7 0x8

AES e Modos de Operação

A classe AESHelper foi implementada utilizando a biblioteca criptográfica do python pycryptodome e realiza a criptografia do AES com 5 modos de operação: ECB (Electronic Codebook), CBC (Cipher Block Chaining), CFB (Cipher Feedback), OFB (Output Feedback) e CTR (Counter).

Para analisar o comportamento dos diferentes modos de operação do AES, foi feita uma análise simples comparativa entre cada modo utilizando a mesma mensagem, chave e, nos casos necessários, vetor de inicialização (IV) e nonce. A entropia e o tempo de execução foram medidos, e os modos foram rankeados com base no tempo médio.

```
from Crypto.Cipher import AES
from Crypto.Util import Counter
from Crypto.Util.Padding import pad

class AESHelper:
    def __init__(self, key_size=16):
        self.key_size = key_size

    # Verifica se a chave tem o tamanho correto
    def check_key_size(self, key: bytes):
        if len(key) != self.key_size:
            raise ValueError(
                f"Key does not match the expected size for AES-
                {self.key_size * 8}")

    # Função principal para criptografar, seleciona o modo de operação
    def encrypt(self, plaintext: bytes, key: bytes, iv: bytes = None,
mode="ECB", nonce: bytes = None) -> bytes:
        self.check_key_size(key)

        if mode == "ECB":
            return self.aes_ecb_encrypt(pad(plaintext, 16), key)
        elif mode == "CBC":
            if iv is None:
                raise ValueError("IV is required for CBC mode")
            return self.aes_cbc_encrypt(pad(plaintext, 16), key, iv)
        elif mode == "CFB":
            if iv is None:
                raise ValueError("IV is required for CFB mode")
            return self.aes_cfb_encrypt(plaintext, key, iv)
        elif mode == "OFB":
            if iv is None:
                raise ValueError("IV is required for OFB mode")
            return self.aes_ofb_encrypt(plaintext, key, iv)
        elif mode == "CTR":
            if nonce is None:
                raise ValueError("Nonce is required for CTR mode")
            return self.aes_ctr_encrypt(plaintext, key, nonce)
```

```

        raise ValueError(f"Unsupported AES mode: {mode}")

    # Criptografia no modo ECB (Electronic Codebook)
    def aes_ecb_encrypt(self, plaintext: bytes, key: bytes) -> bytes:
        cipher = AES.new(key, AES.MODE_ECB)
        return cipher.encrypt(plaintext)

    # Criptografia no modo CBC (Cipher Block Chaining)
    def aes_cbc_encrypt(self, plaintext: bytes, key: bytes, iv: bytes)
-> bytes:
        cipher = AES.new(key, AES.MODE_CBC, iv)
        return cipher.encrypt(plaintext)

    # Criptografia no modo CFB (Cipher Feedback)
    def aes_cfb_encrypt(self, plaintext: bytes, key: bytes, iv: bytes)
-> bytes:
        cipher = AES.new(key, AES.MODE_CFB, iv)
        return cipher.encrypt(plaintext)

    # Criptografia no modo OFB (Output Feedback)
    def aes_ofb_encrypt(self, plaintext: bytes, key: bytes, iv: bytes)
-> bytes:
        cipher = AES.new(key, AES.MODE_OFB, iv)
        return cipher.encrypt(plaintext)

    # Criptografia no modo CTR (Counter)
    def aes_ctr_encrypt(self, plaintext: bytes, key: bytes, nonce:
bytes) -> bytes:
        ctr = Counter.new(64, prefix=nonce, initial_value=0)
        cipher = AES.new(key, AES.MODE_CTR, counter=ctr)
        return cipher.encrypt(plaintext)

```

Para analisar cada modo de operação, foi feito o cálculo do tempo médio e da entropia de cada modo. A **entropia** dos primeiros ciphertexts de cada modo foi calculada usando a fórmula de Shannon:

$$H(X) = - \sum p(x_i) \log_2 p(x_i)$$

Onde $p(x_i)$ é a frequência relativa de cada byte no ciphertext. Essa métrica indica o grau de aleatoriedade do texto cifrado, o que significa que quanto mais próxima de 8 bits/byte (o valor máximo para dados de 8 bits), maior a dispersão e menor a chance de padrões serem gerados.

Já o ranking médio é calculado medindo o tempo que cada modo do AES leva para cifrar o texto em várias execuções, classificando-os do mais rápido ao mais lento a cada vez (com ranking 1 para o mais rápido, 2 para o segundo, e assim por diante), somando essas posições ao longo de todas as execuções e, por fim, dividindo essa soma pelo número total de execuções para obter uma média, que indica o desempenho geral relativo de cada modo, dessa forma quanto menor a média, melhor o desempenho.

```

import os, time
from collections import Counter as Cntr
import math

# Calcula a entropia de Shannon de um array de bytes
def calc_entropy(data: bytes) -> float:
    counter = Cntr(data)
    total = len(data)
    entropy = 0.0
    for count in counter.values():
        p_x = count / total
        entropy -= p_x * math.log2(p_x)
    return entropy

# Lê o arquivo de texto de exemplo
with open("example.txt", "rb") as f:
    plaintext = f.read()

# Gera chave, IV e nonce aleatórios
key = os.urandom(16)
iv = os.urandom(16)
nonce = os.urandom(8)

helper = AESHelper()

modes = ["ECB", "CBC", "CFB", "OFB", "CTR"]
ranking_sums = {mode: 0 for mode in modes}
num_execs = 1000

# Executa o teste de desempenho e entropia para cada modo
for exec_num in range(num_execs):
    timings = {}

    for mode in modes:
        start = time.time()
        # Seleciona os parâmetros corretos para cada modo
        if mode in ["CBC", "CFB", "OFB"]:
            ciphertext = helper.encrypt(plaintext, key, iv=iv,
mode=mode)
        elif mode == "CTR":
            ciphertext = helper.encrypt(plaintext, key, nonce=nonce,
mode=mode)
        else:
            ciphertext = helper.encrypt(plaintext, key, mode=mode)
        end = time.time()
        timings[mode] = end - start

    # Na primeira execução, mostra o ciphertext, tempo e entropia
    if exec_num == 0:
        b64 = base64.b64encode(ciphertext).decode()

```

```

        entropy = calc_entropy(ciphertext)
        print(f"Modo de operação : {mode}")
        print(f"Ciphertext (base64): {b64[:25]}...")
        print(f"Tempo de execução : {end - start:.6f} s")
        print(f"Entropia (bits/byte): {entropy:.3f}")
        print("-" * 50)

# Ordena os modos pelo tempo e soma o ranking
sorted_modes = sorted(timings.items(), key=lambda x: x[1])
for rank, (mode, _) in enumerate(sorted_modes, 1):
    ranking_sums[mode] += rank

# Calcula e exibe o ranking médio de cada modo
print("\n== Ranking Médio (menor é melhor) ==")
sorted_ranking = sorted(ranking_sums.items(), key=lambda x: x[1])
for mode, total_rank in sorted_ranking:
    avg_rank = total_rank / num_execs
    print(f"{mode:4}: {avg_rank:.3f}")

Modo de operação : ECB
Ciphertext (base64): bDTUQtQz8+4GcMPMHB069nxf/...
Tempo de execução : 0.000069 s
Entropia (bits/byte): 7.973
-----
Modo de operação : CBC
Ciphertext (base64): U8LZrKggTvEEQ80bIkV/9cGJg...
Tempo de execução : 0.000044 s
Entropia (bits/byte): 7.972
-----
Modo de operação : CFB
Ciphertext (base64): W9y741KRqu9aznquXDg5UcPah...
Tempo de execução : 0.000179 s
Entropia (bits/byte): 7.975
-----
Modo de operação : OFB
Ciphertext (base64): Wxs1IiHScchZCiElubsmmub...
Tempo de execução : 0.000032 s
Entropia (bits/byte): 7.969
-----
Modo de operação : CTR
Ciphertext (base64): k842Lzgz2mlhm/ERhc7q+gWxD...
Tempo de execução : 0.000030 s
Entropia (bits/byte): 7.967
-----

== Ranking Médio (menor é melhor) ==
ECB : 1.023
CTR : 2.024
OFB : 3.020

```

CBC : 3.935
CFB : 4.998

Análise

S-AES com ECB

No modo ECB, espera-se que blocos idênticos de plaintext produzam blocos idênticos de ciphertext quando cifrados com a mesma chave, já que cada bloco é tratado de forma independente. Como pode ser observado no teste feito, ao ser feita a cifra dos textos "will" e "willyan marques will" foram obtidos os ciphertexts

- 0x36127F2D
- 0x36127F2D0A553EA19DE30365F4BEC40B36127F2D

É claro que o primeiro texto cifrado pode ser facilmente identificado no início e no final da segunda cifra gerada, o que demonstra uma fraqueza enorme desse modo de operação.

AES e modos de operação

Após diversas execuções para cifrar o texto no arquivo `example.txt`, o ranking médio dos modos de operação ficou assim:

1. **ECB**
2. **CTR**
3. **OFB**
4. **CBC**
5. **CFB**

O modo **ECB** ficou em primeiro, o que de certa forma é esperado pois nesse modo os blocos podem ser cifrados de forma independente, já que eles são simplesmente divididos e alimentados ao algoritmo. Isso resulta em menos operações para gerar cada bloco e criptografa-los, além da possibilidade de paralelismo, o que faz com que ele seja mais simples e veloz, mas que também seja o menos seguro.

O **CTR** ficou em segundo lugar, o que também faz sentido. Ele também poderia estar em primeiro lugar, já esse modo gera um contador que é cifrado e combinado com o plaintext o que produz um pequeno overhead, mas permite que os blocos sejam processados em paralelo.

O **OFB** apareceu na sequência. Esse modo funciona como uma cifra de fluxo, gerando um keystream ao cifrar repetidamente um valor inicial. Como cada bloco depende do anterior, não dá para paralelizar, mas as operações por bloco são simples, o que explica seu desempenho intermediário.

O **CBC** foi mais lento porque ele depende do resultado do bloco anterior. Cada bloco de plaintext precisa ser combinado com o bloco cifrado anterior antes de passar pela cifra, o que gera mais operações e impede a paralelização, tornando o processo mais demorado.

Por fim, o **CFB** foi o mais lento, o que é comum. Ele tem dependência entre blocos e realiza mais operações intermediárias, além de, em algumas variantes, processar os dados em pedaços menores (por exemplo, bits ou bytes), o que aumenta o custo por bloco.

Nos resultados obtidos, todos os modos apresentaram entropias muito próximas de 8 bits/byte. Isso é esperado e mostra que os ciphertexts têm boa distribuição de bytes e são indistinguíveis de dados aleatórios na maior parte dos casos. No entanto, embora a entropia alta seja um bom sinal, ela não garante a segurança do modo de operação. Por exemplo, o **ECB**, apesar de ter alta entropia, ainda revela padrões do plaintext quando blocos são repetidos, o que o torna inseguro para a maioria dos usos. O **CBC**, **CFB**, **OFB** e **CTR**, além de apresentarem boa entropia, também protegem contra repetição de padrões no plaintext, desde que usados com IVs e nonces corretos e não reutilizados.

Conclusão

O trabalho desenvolvido permitiu compreender de forma prática o funcionamento do S-AES e dos diversos modos de operação do AES, destacando as diferenças de desempenho, segurança e características de cada um. A implementação do S-AES envolveu os princípios fundamentais do AES como as substituições, permutações e operações no campo finito e como elas podem ser aplicadas de forma simplificada para facilitar o estudo do algoritmo de criptografia simétrica.

Na análise dos modos de operação do AES, os experimentos confirmaram as propriedades teóricas de cada modo. O ECB foi o mais rápido, mas também foi o menos seguro por conta da sua incapacidade eliminar padrões. Já os modos CBC, CFB, OFB e CTR apresentaram mais segurança contra ataques que exploram padrões no plaintext por conta das suas estruturas e do uso de IVs e nonces. Além de cada um manter um bom nível de entropia, seus desempenhos variaram conforme esperado: ECB e CTR mais rápidos, OFB intermediário, e CBC e CFB mais lentos devido às suas dependências entre blocos.

Os resultados reforçam a importância de selecionar o modo de operação de acordo com o contexto da aplicação, equilibrando as necessidades de segurança, desempenho e complexidade de implementação. Além disso, os testes mostram que medidas como entropia são úteis para avaliar a aleatoriedade dos dados cifrados, mas sozinhas não implicam a qualidade dos algoritmos utilizados.